程式設計（115-1）
期中考參考解答

解答提供助教：吳帛軒、陳竑齊
國立臺灣大學資訊管理學系

本次考試總分共 105 分，超過 100 分者的成績以 100 計。

第一題

（20 分；每題 2 分）對下列各敘述，若正確請寫「T」，錯誤請寫「F」。請勿寫任何說明。

(a) T

(b) F

(c) T

(d) T

(e) T

(f) F

(g) T

(h) T

(i) F

(j) T

第二題

（20 分；每小題 2 分）請寫出下列 C++ 程式中每一個 `cout` 敘述的輸出結果，並在作答時將每個輸出結果對應到 `cout` 後的註解內的英文字母標記（如 (a)、(b) 等等）。若結果無法預測，請寫「unpredictable」。請假設所需的函式庫和 namespace 都已經被引入了，且程式使用 ASCII 字元編碼。

(a) 67

(b) C

(c) 0

(d) unpredictable

(e) 3

(f) 22

(g) 1

(h) -1

(i) 3

(j) 2

第三題

（35 分；每小題 5 分）請簡答下列問題。答案需精確，但不必太長。為了避免大家煩惱要敘述得多細，每小題都限 200 字，請大家挑重點寫即可。

參考解答：

(a) 當靜態陣列傳入函數時，會退化成指標，因此在函數內使用 sizeof 只會得到該指標變數本身的大小，無法求出原陣列的長度。

(b) 壞處：會佔用較多記憶體空間，因為呼叫 Stack （堆疊）不斷疊加，有發生 Stack Overflow 的風險，且若未優化可能產生大量重複計算。

適合場景：

1. 適合處理重複性高的結構或演算法，例如：樹、圖等。
2. 問題能被拆解成更小的問題，由解決小問題來解決大問題的時候。

具體案例：費波那契數列求值、河內塔。

評分規則：

- 未寫出壞處或不正確，扣三分
- 未寫出適合的場景（理論或案例皆可，但至少要寫出其中一個），扣兩分

(c) 13

(d) 迴圈版本可寫成：

```
int weirdRep(int n) {
    if (n == 0) return 2;
    if (n == 1) return 1;
    int prev2 = 2, prev1 = 1, current;
    for (int i = 2; i <= n; i++) {
        current = 3 * prev1 - prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return current;
}
```

評分規則：

- 未正確處理 $n = 0$ 或 $n = 1$ 的情況，扣兩分。
- 未寫出計算 $n \geq 2$ 時的計算邏輯或錯誤，扣三分。
- 其餘錯誤，扣一分。

(e) 方法二包含一次排序、k 次二分搜尋，因此總體時間複雜度為 $O(n \log n + k \log n)$ 。因此方法二會在 $n \log n + k \log n < kn$ 時比方法一有效率
將不等式整理一下，得 $n \log n < k(n - \log n)$，當 n 夠大時，n 會遠大於 $\log n$（$n - \log n \approx n$），所以上式可近似為：$n \log n < kn$，兩邊同除 n 得出條件：$k > \log n$

另解：當 k 遠大於 $\log n$，也就是 $k = \omega(\log n)$ 時，方法二的漸進時間複雜度優於方法一。粗略的比較門檻可以直接推導說大約在 $k > \log n$ 時，方法二開始具有優勢。

評分規則：

- 未分析出方法二的總體時間複雜度或是分析錯誤，扣三分
- 未分析 k 所需條件或分析錯誤，扣兩分

(f) 執行最後一行會存取到已被釋放的記憶體空間。這種問題特別危險是因為程式不一定會立刻報錯，而是可能在背景默默覆蓋到其他正在使用的資料，導致錯誤在難以預測的時機點發生，因此難以重現與除錯。

(g) 可能的原因包括：

1. 在不同的作業系統或編譯器架構下，`int` 所佔用的記憶體空間（以及其最大值）可能不同。使用 `INT_MAX` 能確保程式跨平台時的正確性與可攜性
2. 可讀性較高
3. 後續較好維護

評分規則：

- 上述原因至少寫出其中一個即不扣分。

第四題

（20 分；每小題 10 分）請為以下各小題撰寫一個虛擬碼（pseudocode），並為其推導時間複雜度並且以 Big-O 的形式呈現。你的虛擬碼只要是對的（可以得到正確答案、每個步驟都明確地可被執行），就能得到基本分數；如果它是對的但時間複雜度過高，則會被酌予扣分。如果你不會寫虛擬碼，也可以寫 C++ 程式。

參考解答：

(a)

```
Function CountTInS(s, t, n, m):
    // 初始化兩個陣列來紀錄字母出現頻率
    freqS[0..25] = {0}
    freqT[0..25] = {0}

    // 計算字串 s 中每個字元的出現次數
```

```
    for i from 0 to n-1:
        charIndex = s[i] - 'a'
        freqS[charIndex] = freqS[charIndex] + 1

    // 計算字串 t 中每個字元的出現次數
    for i from 0 to m-1:
        charIndex = t[i] - 'a'
        freqT[charIndex] = freqT[charIndex] + 1

    // 找出最多能組出的份數
    maxCopies = 無限大
    for i from 0 to 25:
        if freqT[i] > 0:
            copies = floor(freqS[i] / freqT[i]) // 向下取整
            if copies < maxCopies:
                maxCopies = copies

    return maxCopies
```

時間複雜度推導：遍歷 s 需 $O(n)$，遍歷 t 需 $O(m)$，最後比對 26 個英文字母需 $O(26)$，總時間複雜度為 $O(n+m+26)$，即 $O(n+m)$

評分規則：

- 未寫出能得到正確答案的程式或是過程有誤，扣五分
- 未正確計算時間複雜度，扣三分
- 時間複雜度過高，扣兩分
- 其餘錯誤，扣一分

(b)

```
Function CountNodesAtDistanceTwo(A, v, n):
    // 陣列索引通常從 0 開始，將節點編號 v 轉為索引
    v_idx = v - 1

    // 用來標記哪些節點距離恰好為二，避免重複計算
    isDistanceTwo[0..n-1] = {false}
    count = 0

    // 遍歷所有節點，尋找 v 的直接鄰居
    for i from 0 to n-1:
        if A[v_idx][i] == 1:
            // i 是 v 的鄰居（距離為一），接著找 i 的鄰居 j
            for j from 0 to n-1:
                // 若 j 跟 i 相連，且 j 不是 v 本身，且 j 不與 v 直
                    接相連
```

```
                    if A[i][j] == 1 and j != v_idx and A[v_idx][j] ==
                        0:
                        // 找到了距離恰好為二的節點，標記起來
                        isDistanceTwo[j] = true

        // 統計被標記的節點數量
        for k from 0 to n-1:
            if isDistanceTwo[k] == true:
                count = count + 1

        return count
```

時間複雜度推導：尋找鄰居的鄰居使用了雙層迴圈，外層最多執行 n 次，內層執行 n 次，加上最後一維陣列的遍歷 $O(n)$，總時間複雜度為 $O(n^2)$。

評分規則：

- 未寫出能得到正確答案的程式或是過程有誤，扣五分
- 未正確計算時間複雜度，扣三分
- 時間複雜度過高，扣兩分
- 其餘錯誤，扣一分

第五題

（10 分；每小題 5 分）

參考解答：

(a)

```
int gcd(int a, int b){
    if(a % b == 0) return b;
    return gcd(b, a % b);
}

struct Fraction{
    int num;
    int den;

    void simplify(){
        int divisor = gcd(num, den);
        num /= divisor;
        den /= divisor;
    }
```

```
        int quotient() const {
            return num / den;
        }
    };
```

評分規則：

- 未能正確定義成員變數（member variables），扣兩分
- 未能正確定義成員函數（member functions），扣兩分
- 其餘錯誤（可讀性，如排版、縮排、變數命名方式太差），扣一分

(b)

```
    int numOfSumOfFrac(const Fraction& a, const Fraction& b){
        Fraction sum;

        sum.num = a.num * b.den + b.num * a.den;
        sum.den = a.den * b.den;
        sum.simplify();

        return sum.num;
    }
```

實作為成員函數的原因可能包含：

1. 將加法視為「目前的物件提供了一個與其他同類物件互動的功能」。
2. 讓所有的操作都集中定義在資料型態內部。

實作為全域函數的原因可能包含：

1. 因為是兩個 Fraction 的相加，而非單個 Fraction 的操作

評分規則：

- 未能正確實作函數或實作有誤，扣三分
- 未解釋設計為成員函數還是全域函數的原因，扣兩分
- 其餘錯誤，扣一分